为什么需要 LangGraph

- 来源: https://class.imooc.com/lesson/5284#mid=105034
- 阶段: 阶段四: 【框架应用】高效AI开发
- 周: 第9周 LangGraph：工作流编排
- 模块: 为什么需要LangGraph
- 课节: 图文 1-1 为什么需要 LangGraph

1-1 为什么需要 LangGraph

为什么需要 LangGraph

> [!TIP] 阅读指南 第 8 章我们用 LangChain 构建了简单的 Agent，它能自主决策调用工具。但当任务变得复杂时，单轮推理的黑盒模式就暴露了局限。本节从一个失败的实验开始，带你理解为什么需要 LangGraph 这种更强大的工作流编排工具。

简单 Agent 的局限性

第8章我们学会了用LangChain 构建Agent——它能自主决策调用工具，看起来很智能。但当我让它完成一个稍微复杂的任务时，问题就暴露了。

我想让Agent帮我写一篇技术博客，主题是"Python异步编程"。按第8章学的方法:

```
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def search_info(query: str) -> str:
    """搜索技术资料"""
    # 模拟搜索
    return f"关于{query}的资料..."

@tool
def write_content(topic: str) -> str:
    """生成文章内容"""
    # 调用LLM生成
    return f"关于{topic}的文章..."

agent = create_agent(model=llm, tools=[search_info, write_content])
result = agent.invoke({"messages": [{"role": "user", "content": "写一篇关于Python异步编程的
博客"}]})

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

预览
```

结果看起来似乎智能，但问题很明显：Agent 可能调用 search_info 搜索资料，然后调用 write_content 生成文章。但仔细看，问题恰恰出在它的"一次性"上——所有决策在一次对话中完成，无法暂停审查、无法分阶段确认、中间状态全部丢失、流程完全不可控。

核心问题: Agent虽然能自主决策，但它是一口气执行完,而不是分步骤、可暂停、可干预的工作流。

真实场景的需求

实际项目中，我们需要的不是"一口气执行完",而是结构化的多阶段工作流:

结构化的多阶段工作流

场景1:内容创作

```
1. 搜集资料(调用搜索工具)
   ↓
   [暂停] → 让我审查资料是否相关
   ↓
2. 生成大纲(LLM规划结构)
   ↓
   [暂停] → 让我确认大纲是否合理
   ↓
3. 分段扩写(逐段生成内容)
   ↓
4. 润色优化(语言打磨)
   ↓
5. 质量检查(验证可读性)
```

关键需求:

关键需求

- 需要多个步骤顺序执行

多个步骤

- 需要中间状态传递(如大纲传给扩写步骤)
- 需要人工干预(在关键节点暂停确认)

人工干预

- 需要可视化调试(看到每步执行情况)

可视化调试

> 有人可能会说，分步骤太麻烦，让AI一次性搞定不是更好？但这就是关键区别。如果只是随便玩玩，一次性推理确实够用。但要生成高质量、可靠的内容，现阶段AI还做不到完全不需要人工干预，分步骤检查是更务实的做法。

问题的本质：黑盒推理 vs. 白盒流程

为什么简单 Agent 无法胜任复杂任务？本质在于控制权的缺失。

控制权的缺失

在第 8 章中，我们通过 create_agent 构建了"自动挡" Agent，它内部如何跳转、何时调用工具完全由模型"拍脑袋"决定。为了看清两者的本质差异，我们对比它们处理同一个任务时的"思维路径"：

```
【 简单 Agent】
执行模式：黑盒一次性推理 (类似“高铁”，中途不可停靠、不可换乘)
路径：用户指令 -> [ LLM 内部决策 ] -> 最终输出
状态：隐式（仅靠 Context 维持，每步不可控）

【 LangGraph 工作流】
执行模式：白盒分阶段执行 (类似“自驾”，每到路口可导航、可随时休息审核)
路径：用户指令 -> 节点 A (大纲) -> [ 暂停审核 ] -> 节点 B (扩写) -> 最终输出
状态：显式（数据持久化在 State 对象中，每步可查）
```

上面是"黑盒一次性执行"，下面是"白盒分阶段执行"，两者的本质差异在于控制权——前者完全由模型随机决定，后者每个步骤都显式可控。

LangGraph 的完整处理流程

所以，LangGraph 就是为了解决这个问题而生的。 LangGraph 把 AI 从"黑盒的自动决策机器"变成了"白盒的分阶段执行流程"。

让我们通过一个完整的流程图，看看 LangGraph 如何处理“写文章”这个任务：

关键特点体现在四个方面：

- 状态驱动：每个阶段都有明确的状态定义（初始、已搜集资料、已有大纲...）。

状态驱动

- 数据累积：State 对象不断累积新字段，保留所有中间结果。

数据累积

- 流程可控：每一步转换都是显式定义的，不依赖模型随机决策。

流程可控

- 可暂停恢复：可以在任意节点暂停检查，修改后继续执行。

可暂停恢复

LangGraph 的五大典型应用场景

理解了 LangGraph 的技术优势后，我们来看看它在实际项目中能解决哪些问题。

场景 1：多步骤任务编排（数据分析流水线）

假设你要开发一个"自动数据分析助手"，完整流程是：上传数据 → 清洗异常值 → 统计分析 → 生成可视化图表 → 输出报告。

- 简单 Agent 的问题：模型可能直接跳到"生成报告"，跳过清洗步骤；或者清洗后的数据没保存，分析节点拿到的是脏数据。
- LangGraph 的解决方案：每个步骤（清洗、分析、绘图、报告）都是独立的节点，State 对象在流程中不断累积字段（raw_data → cleaned_data → statistics → chart_path → report），确保数据完整传递。

场景 2：条件分支（智能客服路由）

用户问题可能是"查订单"、“退货申请"或"技术咨询”，需要分别调用不同的后端服务。

- 简单 Agent 的问题：模型可能误判问题类型，或者没有明确的分流机制，导致回答文不对题。
- LangGraph 的解决方案：定义一个"分类节点"，根据问题类型返回不同的下一步（如 order_query 或 refund_process），通过 add_conditional_edges 实现动态路由。

场景 3：循环与重试（代码生成验证）

让 AI 生成代码，如果运行出错则自动修复，最多重试 3 次。

- 简单 Agent 的问题：一次生成，要么成功要么失败，无法自动重试；即使能重试，也难以控制次数（可能无限循环）。
- LangGraph 的解决方案：定义"生成代码" → “运行验证” → "判断结果"三个节点，如果验证失败则返回到"修复代码"节点，State 中记录 retry_count 字段，超过 3 次后强制退出。

场景 4：多智能体协作（内容创作团队）

一个专业的"写作助手"由三个 Agent 协作完成：研究员（搜集资料）、编辑（撰写内容）、审核员（质量把关）。

- 简单 Agent 的问题：单个 Agent 既要搜资料又要写文章，能力不够专注；多个 Agent 之间无法共享中间数据。
- LangGraph 的解决方案：三个 Agent 分别作为三个节点，共享同一个 State 对象（包含 materials、draft、review_comments 等字段），通过状态传递实现协作。

场景 5：人工介入（内容审核助手）

AI 自动生成社交媒体推广文案，但在发布前需要人工审核敏感词和品牌调性。

- 简单 Agent 的问题：一旦执行就无法暂停，要么全自动（可能发布不当内容）要么全人工（失去效率优势），无法实现"半自动"模式。
- LangGraph 的解决方案：在"生成文案"节点后设置 interrupt_before，暂停流程等待人工审核，审核人员可以在 State 中修改文案内容或标记"通过"，然后调用 resume() 继续后续的发布流程。

■ 学点英语

中文	English	音标	说明
语言图框架	LangGraph	/læŋ græf/	基于图结构的工作流编排框架，用有向图建模 AI 流程
状态图	StateGraph	/steɪt græf/	LangGraph 的核心类，管理 State + Node + Edge 的容器
工作流编排	Workflow Orchestration	/ˈwɜːrkfloʊ ˌɔːrkɪˈstreɪʃn/	将多个 AI 步骤按规则组合成可控流程
黑盒 vs 白盒	Black Box vs White Box	/blæk bɑːks ˈvɜːrsəs waɪt bɑːks/	不可控一次性执行 vs 分阶段可干预执行

下节预告

下一节，我们将深入学习 LangGraph 的核心概念：State（状态）、Node（节点）、Edge（边） 和 Graph（图），并动手构建第一个状态图工作流。

图片

